



Guía de Despliegue en Kubernetes

AsambleasGiramaster — Sistema de Administración de Unidades Residenciales con Reuniones Virtuales Integradas

Índice

1. Arquitectura general

2. Prerrequisitos

Instalar el Ingress Controller

3. Estructura de los manifiestos

4. Despliegue inicial (manual)

4.1 Construir las imágenes

4.2 Aplicar los manifiestos

5. Despliegue automático (CI/CD)

5.1 Qué hace el workflow, paso a paso

5.2 Disparar el workflow manualmente

6. Configuración de DNS y TLS

6.1 Obtener la IP del Ingress

6.2 Configurar el dominio (asambleas.giramaster.co)

6.3 Verificar el certificado SSL

7. Verificación post-despliegue

Health checks por componente

8. Logs y monitoreo

9. Escalado

10. Actualizaciones

Camino recomendado: push a master

Camino manual (si se necesita fuera del pipeline)

11. Rollback

12. Celery (workers)

13. Troubleshooting común

El Ingress no obtiene IP externa

El SSL no funciona

Un pod no inicia

14. Limpieza / desinstalación

15. Variables de entorno y secretos

Backend — ConfigMap (backend-config)

Backend — Secret (backend-secret)

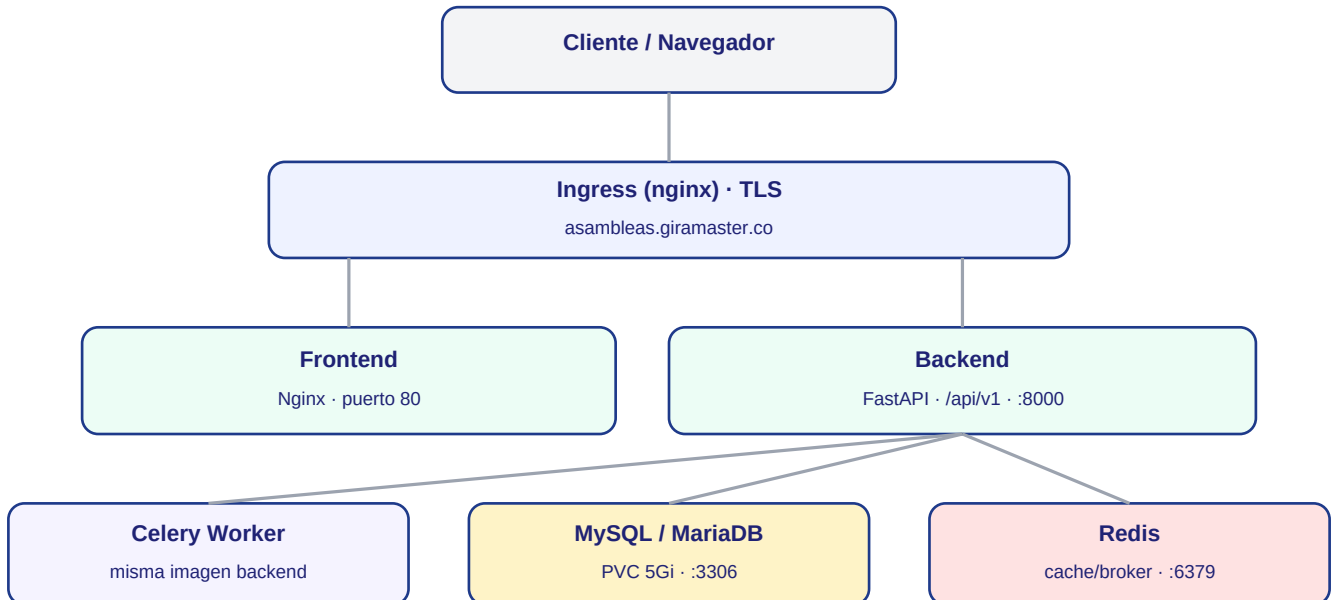
Frontend — ConfigMap (frontend-config)

MySQL — Secret (mysql-secret)

16. Configuración de recursos por componente

1. Arquitectura general

AsambleasGiramaster se despliega como un conjunto de servicios independientes dentro de un cluster de Kubernetes (k3s), coordinados por manifiestos de Kustomize ubicados en `k8s/base/` y el overlay `k8s/overlays/local/`. Todos los recursos viven en el namespace `default`.



- **Ingress (nginx-ingress):** único punto de entrada HTTPS, enruta `/` al frontend y `/api/v1` al backend. TLS vía certificado Cloudflare Origin CA.
- **Frontend:** build estático de React servido por Nginx (puerto 80), 2 réplicas.
- **Backend:** API FastAPI (puerto 8000), 2 réplicas.
- **Celery Worker:** misma imagen Docker que el backend, seleccionada vía variable de entorno `SERVICE=celery`, procesa la cola `email_tasks`.
- **MySQL / MariaDB 10.11:** 1 réplica con almacenamiento persistente (PVC de 5Gi).
- **Redis 7:** caché de sesiones y broker/result-backend de Celery, sin persistencia.

2. Prerrequisitos

Antes de desplegar, el nodo/cluster debe contar con:

- Un cluster de Kubernetes accesible (k3s en producción).
- `kubectl` configurado y apuntando al cluster (`~/ .kube/config` o `KUBECONFIG`).
- Docker, para construir las imágenes del backend y del frontend.
- Un Ingress Controller instalado (se usa `nginx-ingress`).

Instalar el Ingress Controller

```
# Minikube
minikube addons enable ingress

# K3s: ya viene incluido por defecto (Traefik) – si se prefiere nginx:
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/\
controller-v1.9.4/deploy/static/provider/cloud/deploy.yaml

# Kind
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/\
main/deploy/static/provider/cloud/deploy.yaml
```

3. Estructura de los manifiestos

Los manifiestos se organizan con Kustomize: una capa base/ con los recursos genéricos y un overlay overlays/local/ que fija el namespace.

```
k8s/
|-- base/
|   |-- frontend/
|   |   |-- configmap.yaml    # VITE_API_URL
|   |   |-- deployment.yaml   # imagen giramaster-frontend:latest, 2 replicas
|   |   |-- secret.yaml       # variables sensibles del frontend (si las hay)
|   |   |-- service.yaml      # ClusterIP :80
|   |-- backend/
|   |   |-- configmap.yaml     # HOST_DB, REDIS_HOST, ENVIRONMENT, etc.
|   |   |-- deployment.yaml   # imagen giramaster-backend:latest, 2 replicas
|   |   |-- secret.yaml       # SECRET_KEY, credenciales de BD, FRONTEND_URL...
|   |   |-- service.yaml      # ClusterIP :8000
|   |-- celery/
|   |   |-- deployment.yaml    # misma imagen que backend, SERVICE=celery
|   |-- redis/
|   |   |-- configmap.yaml     # redis.conf (maxmemory, allkeys-lru)
|   |   |-- deployment.yaml    # redis:7-alpine + Service :6379
|   |-- mysql/
|   |   |-- configmap.yaml
|   |   |-- deployment.yaml    # mariadb:10.11, strategy Recreate
|   |   |-- pvc.yaml           # 5Gi ReadWriteOnce
|   |   |-- secret.yaml        # MYSQL_ROOT_PASSWORD, MYSQL_USER, ...
|   |   |-- service.yaml       # ClusterIP :3306
|   |-- ingress.yaml           # host, TLS, reglas /api/v1 y /
|   |-- tls-secret.yaml        # certificado TLS (Cloudflare Origin CA)
|   |-- kustomization.yaml
|-- overlays/
|   |-- local/
|   |   |-- kustomization.yaml # namespace: default
```

4. Despliegue inicial (manual)

Para el primer despliegue en un cluster nuevo (o para recrear el entorno desde cero), el flujo es completamente manual con `kubectl` + Kustomize.

4.1 Construir las imágenes

```
# Backend (incluye también celery-worker y celery-beat, seleccionados por SERVICE)
docker build -t giramaster-backend:latest ./backend

# Frontend: primero se compila el build de React, luego la imagen Nginx
cd frontend
pnpm install --frozen-lockfile
pnpm build
cd ..
docker build -t giramaster-frontend:latest ./frontend
```

Nota: Los Deployments usan `imagePullPolicy: Never`: Kubernetes nunca intenta descargar la imagen de un registry, espera que ya exista en el nodo. Si el cluster tiene varios nodos, hay que construir (o importar) la imagen en cada uno donde pueda programarse el pod.

4.2 Aplicar los manifiestos

```
# Desde la raíz del proyecto
kubectl apply -k k8s/overlays/local

# Verificar que todo quedó aplicado
kubectl get all -n default
```

Ejemplo ilustrativo de la salida esperada (los nombres de pod varían en cada despliegue):

NAME	READY	STATUS	RESTARTS	AGE
pod/backend-6c9b7d4f9c-2xk7q	1/1	Running	0	45s
pod/backend-6c9b7d4f9c-9mvwl	1/1	Running	0	45s
pod/frontend-7f8d9c5b6d-4hqjw	1/1	Running	0	45s
pod/frontend-7f8d9c5b6d-8pzrm	1/1	Running	0	45s
pod/celery-worker-5d8f7c9b6-k2n4t	1/1	Running	0	45s
pod/celery-worker-5d8f7c9b6-t8w3x	1/1	Running	0	45s
pod/mysql-5f6d8c9b7-lq9zv	1/1	Running	0	45s
pod/redis-7b8c9d6f5-w4m2p	1/1	Running	0	45s

NAME	TYPE	CLUSTER-IP	PORT(S)	AGE
service/backend	ClusterIP	10.43.120.11	8000/TCP	45s
service/frontend	ClusterIP	10.43.88.203	80/TCP	45s
service/mysql	ClusterIP	10.43.55.140	3306/TCP	45s
service/redis	ClusterIP	10.43.201.9	6379/TCP	45s

5. Despliegue automático (CI/CD)

Para el día a día, el despliegue real ocurre a través de un workflow de GitHub Actions (`.github/workflows/deploy.yml`) que se dispara automáticamente con cada push a la rama master, o manualmente desde la pestaña *Actions* del repositorio (`workflow_dispatch`).

5.1 Qué hace el workflow, paso a paso

El workflow se conecta por SSH al servidor de producción y ejecuta 6 pasos, todos dentro de `/srv/proyectos/AsambleasGiramaster`:

- **[1/6] Actualizar código:** `git fetch origin master && git reset --hard origin/master`.
- **[2/6] Build frontend:** `pnpm install --frozen-lockfile && pnpm build` (se puede omitir con el `input skip_frontend_build`).

- **[3/6] Build de imágenes Docker:** `docker build --no-cache` para `giramaster-backend` y `giramaster-frontend`, ambas etiquetadas también con el SHA corto del commit.
- **[4/6] Importar a k3s:** `docker save ... | sudo k3s ctr images import -` — no se usa ningún registry externo, la imagen se inyecta directo al containerd del nodo.
- **[5/6] Rollout restart:** reinicia backend, frontend y celery-worker.
- **[6/6] Verificación:** `kubectl rollout status` de los 3 Deployments con timeout de 120s, y resumen final con el estado de los pods.

Salida real que imprime el workflow en cada corrida (recortada):

```
=====
GIRAMASTER - Deploy 2026-07-29 20:10:03
=====

[1/6] Actualizando código desde master...
✓ Código actualizado → a1b2c3d fix: ...

[2/6] Compilando frontend...
✓ Frontend compilado (dist/)

[3/6] Construyendo imágenes Docker...
→ Backend...
✓ giramaster-backend:latest
→ Frontend...
✓ giramaster-frontend:latest

[4/6] Importando imágenes a k3s...
→ Importando backend...
✓ Backend importado
→ Importando frontend...
✓ Frontend importado

[5/6] Reiniciando deployments...
✓ Restart solicitado a los 3 deployments

[6/6] Esperando que los pods estén Ready...
✓ backend OK
✓ frontend OK
✓ celery-worker OK

=====
✓ Deploy exitoso
=====
```

5.2 Disparar el workflow manualmente

Desde GitHub → pestaña *Actions* → workflow *Deploy to Production* → *Run workflow*. Permite marcar `skip_frontend_build: true` cuando solo cambió el backend, para ahorrar tiempo de build.

6. Configuración de DNS y TLS

6.1 Obtener la IP del Ingress

```
kubectl get ingress main-ingress -n default \
-o jsonpath='{.status.loadBalancer.ingress[0].ip}'
```

6.2 Configurar el dominio (asambleas.giramaster.co)

- En el proveedor DNS (Cloudflare), crear/editar un registro **A** apuntando a la IP del Ingress.
- **Name:** subdominio correspondiente a `asambleas.giramaster.co`.
- **Proxy status:** activado (orange cloud), si se usa Cloudflare como proxy/CDN.

Nota: El host configurado en `k8s/base/ingress.yaml` y en `k8s/base/tls-secret.yaml` debe coincidir exactamente con el dominio real (`asambleas.giramaster.co`) y con el certificado TLS emitido para ese dominio.

6.3 Verificar el certificado SSL

```
curl -vI https://asambleas.giramaster.co/

openssl s_client -connect asambleas.giramaster.co:443 -servername asambleas.giramaster.co
```

7. Verificación post-despliegue

Servicio	URL
Frontend	<code>https://asambleas.giramaster.co/</code>
Backend API	<code>https://asambleas.giramaster.co/api/v1/</code>
Health (frontend)	<code>https://asambleas.giramaster.co/health</code>

Health checks por componente

```
# MySQL / MariaDB
kubectl get pods -l app=mysql
kubectl exec -it <mysql-pod> -- mysqladmin ping

# Backend
curl https://asambleas.giramaster.co/api/v1/

# Frontend
curl https://asambleas.giramaster.co/health
```

8. Logs y monitoreo

```
kubectl logs -l app=frontend -f
kubectl logs -l app=backend -f
kubectl logs -l app=celery-worker -f
kubectl logs -l app=mysql -f
kubectl logs -l app=redis -f

# Eventos recientes del cluster
kubectl get events --sort-by='.lastTimestamp'
```

9. Escalado

```
kubectl scale deployment frontend      --replicas=3
kubectl scale deployment backend      --replicas=2
kubectl scale deployment celery-worker --replicas=4
```

10. Actualizaciones

Camino recomendado: push a master

Basta con fusionar los cambios a master; el CI/CD hace el resto (sección 5).

Camino manual (si se necesita fuera del pipeline)

```
# Backend (también reinicia celery-worker, comparte imagen)
docker build -t giramaster-backend:latest ./backend
kubectl rollout restart deployment/backend
kubectl rollout restart deployment/celery-worker
kubectl rollout status deployment/backend
kubectl rollout status deployment/celery-worker

# Frontend
cd frontend && pnpm build && cd ..
docker build -t giramaster-frontend:latest ./frontend
kubectl rollout restart deployment/frontend
```

11. Rollback

```
kubectl rollout history deployment/backend
kubectl rollout undo deployment/backend
kubectl rollout undo deployment/backend --to-revision=2
```

12. Celery (workers)

```
# Ver pods y logs
kubectl get pods -l app=celery-worker
kubectl logs -l app=celery-worker -f

# Estado de la cola
kubectl exec -it <celery-pod> -- celery -A app.celery_app inspect active
kubectl exec -it <celery-pod> -- celery -A app.celery_app inspect ping
kubectl exec -it <celery-pod> -- celery -A app.celery_app inspect scheduled
kubectl exec -it <celery-pod> -- celery -A app.celery_app inspect reserved

# Escalar workers
kubectl scale deployment celery-worker --replicas=4
```

13. Troubleshooting común

El Ingress no obtiene IP externa

```
kubectl describe ingress main-ingress
kubectl get svc -n ingress-nginx
```

El SSL no funciona

```
kubectl describe secret tls-secret
kubectl describe ingress main-ingress
```

Un pod no inicia

```
kubectl logs <pod-name>
kubectl describe pod <pod-name>
```

14. Limpieza / desinstalación

```
# Eliminar todos los recursos del overlay
kubectl delete -k k8s/overlays/local

# Eliminar el PVC (¡CUIDADO! Borra los datos de MySQL)
kubectl delete pvc mysql-pvc

# Eliminar imágenes locales (opcional)
docker rmi giramaster-backend:latest giramaster-frontend:latest
```

15. Variables de entorno y secretos

Nota: Esta sección documenta únicamente **nombres y propósito** de las variables. Los valores reales viven en los Secret de Kubernetes (k8s/base/*/secret.yaml) y no se incluyen en este documento por seguridad.

Backend — ConfigMap (backend-config)

Variable	Descripción
ENVIRONMENT	development / production.
HOST, PORT	Bind de Uvicorn dentro del contenedor.
HOST_DB, PORT_DB, NAME_DB	Conexión a MySQL/MariaDB (host = Service "mysql").
REDIS_HOST, REDIS_PORT	Conexión a Redis (host = Service "redis").

Backend — Secret (backend-secret)

Variable	Descripción
SECRET_KEY, REFRESH_SECRET_KEY	Firma de los JWT de sesión y de auto-login/QR.
ENCRYPTION_MASTER_KEY	Cifra credenciales sensibles guardadas en BD (ej. integraciones).
USER_DB, PASSWORD_DB	Credenciales de la base de datos.
FRONTEND_URL	Base usada para construir los links de auto-login/QR en emails.
REDIS_URL, CELERY_RESULT_BACKEND	URL de conexión a Redis para Celery.

Frontend — ConfigMap (frontend-config)

Variable	Descripción
VITE_API_URL	Prefijo de la API consumida por el SPA (por defecto "/api/v1").

MySQL — Secret (mysql-secret)

Variable	Descripción
MYSQL_ROOT_PASSWORD	Password del usuario root del contenedor MariaDB.
MYSQL_USER, MYSQL_PASSWORD	Credenciales de la aplicación.
MYSQL_DATABASE	Nombre de la base de datos creada al iniciar el contenedor.

16. Configuración de recursos por componente

Componente	Réplicas	Requests → Limits
Frontend	2	64Mi / 100m → 128Mi / 200m
Backend	2	256Mi / 250m → 512Mi / 500m
Celery Worker	2	256Mi / 250m → 512Mi / 500m
MySQL / MariaDB	1	— (ver deployment.yaml)
Redis	1	— (ver deployment.yaml)